

Московский авиационный институт (национальный исследовательский университет)

Институт «Компьютерные науки и прикладная математика»

Кафедра «Вычислительная математика и программирование»

Лабораторная работа №1

по курсу «Дискретный анализ»

на тему «Сортировки за линейное время»

Студент: Строков Георгий Владимирович

Группа: М8О-208БВ-24

Преподаватель: Макаров Н.К.

Проверил: Сахарин Н.А.

Дата: 21 марта 2026

Оценка: _____

Подпись: _____

Москва, 21 марта 2026

1 Постановка задачи

Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа указанным алгоритмом сортировки за линейное время и вывод отсортированной последовательности. Алгоритм сортировки: поразрядная сортировка.

2 Формат ввода

На каждой непустой строке входного файла располагается пара «ключ-значение», в которой ключ указан согласно заданию, затем следует знак табуляции и указано соответствующее значение. Каждая строка содержит:

- Тип ключа: даты в формате DD.MM.YYYY.
- Тип значения: числа от 0 до $2^{64}-1$.

Пары разделены пробельными символами.

3 Формат вывода

Выходные данные состоят из тех же строк, что и входные, за исключением пустых и порядка следования. Порядок следования определяется алгоритмом сортировки, согласно условию задания.

4 Метод решения (Алгоритм)

Используется алгоритм **Radix sort** (поразрядная сортировка) с битовой разрядностью 11. Ключом сортировки является дата, преобразованная в целое число по формуле:

$$date = year \times 12 \times 31 + month \times 31 + day,$$

что обеспечивает полный порядок дней от 1 января до 31 декабря включительно. Алгоритм состоит из следующих шагов:

1. Создаётся объект **Digitizer**, который определяет разрядность типа ключа (32 бита).
2. Создаётся объект **Order**, который управляет текущим сдвигом (индексом) и шагом (11 бит). На каждой итерации из ключа выделяется 11-битный фрагмент.
3. Для текущего фрагмента выполняется **counting sort**:
 - Подсчитывается количество элементов для каждого возможного значения фрагмента (размер буфера $2^{11} = 2048$).
 - Вычисляются префиксные суммы для определения позиций.

- Элементы переставляются во вспомогательный буфер (или обратно) с использованием устойчивой сортировки.
4. Шаг повторяется для следующих фрагментов (сдвиг увеличивается на 11), пока не будут обработаны все 32 бита. Всего выполняется $\lceil 32/11 \rceil = 3$ итерации.
 5. После завершения всех итераций данные оказываются отсортированными по возрастанию ключа.

Важной особенностью реализации является чередование сортировки из исходного массива во вспомогательный и обратно, что позволяет избежать дополнительного копирования на каждом проходе. В конце, если последняя сортировка поместила данные в буфер, выполняется финальное копирование в исходный массив.

5 Исходный код

```
1  #include <iostream>
2  #include <c std def>
3  #include <c std int>
4  #include <iterator>
5  #include <limits>
6
7
8
9  struct Item {
10     uint32_t date;
11     uint32_t pos;
12     uint32_t len;
13 };
14
15
16
17 template <typename T>
18 class MyVector {
19 private:
20     T* data_;
21     std :: size _t size_;
22     std :: size _t capacity_;
23
24 public:
25     MyVector()
26     : data_{new T[8]()}
27     , size_{0}
```

```

28     , capacity_{8}
29     {}
30
31     explicit MyVector( std :: size _t n)
32     : data_{new T[n]() }
33     ,   size_{n}
34     , capacity_{n}
35     {}
36
37     template <typename Iter>
38     MyVector(Iter start, Iter end)
39     : data_{nullptr}
40     ,   size_{0}
41     , capacity_{0}
42     {
43         auto dist = std :: distance(start, end);
44         data_ = new T[dist]();
45         size_ = dist;
46         capacity_ = dist;
47         std :: size _t idx = 0;
48         for (auto it = start; it != end; ++it) {
49             data_[idx++] = *it;
50         }
51     }
52
53     MyVector(const MyVector& other)
54     : data_{new T[other.capacity_] }
55     ,   size_{other.size_}
56     , capacity_{other.capacity_}
57     {
58         for (std :: size _t i = 0; i != other.size_; ++i) {
59             data_[i] = other.data_[i];
60         }
61     }
62
63     MyVector(MyVector&& other) noexcept
64     : data_{other.data_}
65     ,   size_{other.size_}
66     , capacity_{other.capacity_}
67     {
68         other.data_ = nullptr;
69         other.size_ = 0;
70         other.capacity_ = 0;

```

```

71     }
72
73     MyVector() noexcept {
74         delete[] data_;
75         size_ = 0;
76         capacity_ = 0;
77     }
78
79     MyVector& operator=(const MyVector& other) {
80         if (this != &other) {
81             delete[] data_;
82
83             data_ = new T[other.capacity_];
84             size_ = other.size_;
85             capacity_ = other.capacity_;
86             for (std::size_t i = 0; i != other.size_; ++i) {
87                 data_[i] = other.data_[i];
88             }
89         }
90
91         return *this;
92     }
93
94     MyVector& operator=(MyVector&& other) noexcept {
95         if (this != &other) {
96             delete[] data_;
97
98             data_ = other.data_;
99             size_ = other.size_;
100            capacity_ = other.capacity_;
101
102            other.data_ = nullptr;
103            other.size_ = 0;
104            other.capacity_ = 0;
105        }
106
107        return *this;
108    }
109
110    T& operator[](std::size_t index) { return data_[index]; }
111    const T& operator[](std::size_t index) const { return data_[index]; }
112

```

```

113     std :: size _t size () const { return size _; }
114     std :: size _t capacity () const { return capacity _; }
115
116     void push_back (const T& value) {
117         if ( size _ == capacity _ ) {
118             capacity _ = (capacity _ == 0 ? 1 : capacity _ * 2);
119             T* new_data = new T[capacity _];
120             for ( size _t i = 0; i != size _; ++i) {
121                 new_data[i] = std :: move(data_[i]);
122             }
123
124             delete [] data _;
125             data _ = new_data;
126         }
127
128         data_[ size _++] = value;
129     }
130
131     T* begin () { return data _; }
132     const T* begin () const { return data _; }
133     T* end () { return data _ + size _; }
134     const T* end () const { return data _ + size _; }
135
136     using reverse_iterator = std :: reverse_iterator<T*>;
137     using const_reverse_iterator = std :: reverse_iterator<const T*>;
138
139     reverse_iterator rbegin () { return reverse_iterator(end()); }
140     const_reverse_iterator rbegin () const { return
141         const_reverse_iterator(end()); }
142     reverse_iterator rend () { return reverse_iterator(begin()); }
143     const_reverse_iterator rend () const { return const_reverse_iterator
144         (begin()); }
145
146     void clear () {
147         delete [] data _;
148         data _ = new T[1]();
149         size _ = 0;
150         capacity _ = 1;
151     }
152
153     T* data () { return data _; }
154     const T* data () const { return data _; }
155 };

```

```

154
155
156
157 template <typename T>
158 class Digitizer final{
159 private:
160     std :: size _t width_;
161
162 public:
163     Digitizer()
164     : width_{ std :: numeric_limits<T> :: is_signed + std :: numeric_limits<T>
        :: digits}
165     {
166     }
167
168     template <typename Iterator>
169     std :: size _t get_pos(Iterator it, std :: size _t index, std :: size _t
        step) const {
170         auto value = it->date;
171         return (value » index) & ((T{1} « step) - T{1});
172     }
173
174     std :: size _t get_width() const { return width_; }
175 };
176
177
178 class RadixSort final{
179 private:
180     std :: size _t step_ = 11;
181
182 public:
183     RadixSort() = default;
184     RadixSort( std :: size _t step) : step_{step} {}
185
186 private: //for private classes
187     template <typename Digitizer>
188     class Order {
189     private:
190         std :: size _t index_;
191         std :: size _t step_;
192         Digitizer digit_;
193
194     public:

```

```

195         Order( std :: size _t index, std :: size _t step, Digitizer digit)
196             : index_{index}
197             , step_{step}
198             , digit_{std :: move(digit)}
199         {
200         }
201
202         std :: size _t get_index() const { return index_; }
203         std :: size _t get_width() const { return digit_.get_width();
204             }
205         void update_index() { index_ += step_; }
206         std :: size _t radix() const { return 1 « step_; }
207
208         template <typename Iterator>
209         std :: size _t get_pos(Iterator it) const {
210             return digit_.get_pos(it, index_, step_);
211         }
212     };
213
214 private: //for private funcs
215     template <typename InIter, typename OutIter, typename Digitizer>
216     void counting_sort(
217         InIter start_it, InIter end_it,
218         OutIter out_it, Order<Digitizer> order)
219     {
220         MyVector<std :: size _t> count(order.radix());
221
222         for (auto it = start_it; it != end_it; ++it) {
223             ++count[order.get_pos(it)];
224         }
225
226         for (std :: size _t i = 1; i != order.radix(); ++i) {
227             count[i] += count[i - 1];
228         }
229
230         auto r_begin = std :: reverse_iterator(end_it);
231         auto r_end = std :: reverse_iterator(start_it);
232         for (auto it = r_begin; it != r_end; ++it) {
233             std :: size _t pos = count[order.get_pos(it)] - 1;
234             pos -= count[order.get_pos(it)];
235             out_it += pos;
236             *out_it = *it;

```

```

237         out_it -= pos;
238     }
239 }
240
241 public:
242     template <typename Iterator, typename Digitizer>
243     void radix_sort(Iterator start, Iterator end, Digitizer digit) {
244         if (start == end) { return; }
245
246         Order order(0, step_, digit);
247
248         std::size_t size = static_cast<std::size_t>(std::distance(start
249             , end));
250         using T = typename std::iterator_traits<Iterator>::value_type;
251         MyVector<T> buffer(size);
252
253         bool sorted_in_buffer = false;
254         auto buffer_start = buffer.begin();
255         auto buffer_end = buffer.end();
256
257         while (order.get_index() < order.get_width()) {
258             if (!sorted_in_buffer) {
259                 counting_sort(start, end, buffer_start, order);
260                 sorted_in_buffer = true;
261             } else {
262                 counting_sort(buffer_start, buffer_end, start, order);
263                 sorted_in_buffer = false;
264             }
265
266             order.update_index();
267         }
268
269         if (sorted_in_buffer) {
270             auto buffer_it = buffer_start;
271             for (auto it = start; it != end; ++it) {
272                 *it = *buffer_it++;
273             }
274         }
275     }
276 };
277
278

```

```

279
280 class Parser final {
281 private:
282     template <typename T>
283     static std :: size _t find(const T& line, char c, std :: size _t start =
        0) noexcept {
284         for (std :: size _t i = start; i != line.size (); ++i) {
285             if (line[i] == c) {
286                 return i;
287             }
288         }
289
290         return -1;
291     }
292
293     template <typename Iterator>
294     static int stoi(Iterator start, Iterator end, std :: size _t shift = 0
        ) noexcept {
295         int result = 0;
296         while (start + shift != end) {
297             if (*(start + shift) >= '0' && *(start + shift) <= '9') {
298                 result = result * 10 + *(start + shift) - '0';
299             } else {
300                 return result;
301             }
302
303             ++start;
304         }
305
306         return result;
307     }
308
309 public:
310     template <typename T>
311     static Item parser(const T& line, std :: size _t ind) noexcept {
312         std :: size _t pos1 = find(line, '.');
313         std :: size _t pos2 = find(line, '.', pos1 + 1);
314         std :: size _t pos3 = find(line, '\\t');
315
316         uint8_t day = static_cast<uint8_t>(stoi(line.begin(), line.end(
            )),);
317         uint8_t month = static_cast<uint8_t>(stoi(line.begin() + pos1 +
            1, line.end()));

```

```

318     uint16_t year = static_cast<uint16_t>(stoi(line.begin() + pos2
        + 1, line.end()));
319     uint32_t date = static_cast<uint32_t>(year) * 12 * 31 +
320         static_cast<uint32_t>(month) * 31 +
321         static_cast<uint32_t>(day);
322     uint32_t pos = static_cast<uint32_t>(ind);
323     uint32_t len = static_cast<uint32_t>(line.size());
324
325     return Item{date, pos, len};
326 }
327 };
328
329
330
331 int main() {
332     MyVector<Item> vec;
333     MyVector<char> all_data;
334     uint32_t pos = 0;
335
336     char c;
337     MyVector<char> line;
338     bool line_feed_flag = false;
339     while (std::cin.get(c)) {
340         if (c == '\n') {
341             if (line.size() > 0) {
342                 vec.push_back(Parser::parser(line, pos));
343                 line.clear();
344                 pos = all_data.size();
345             }
346
347             } else {
348                 all_data.push_back(c);
349                 line.push_back(c);
350             }
351         }
352
353         if (line.size() > 0) {
354             vec.push_back(Parser::parser(line, pos));
355             line.clear();
356         }
357
358         Digitizer<std::uint32_t> digit;
359         RadixSort rs;

```

```

360     rs.radix_sort(vec.begin(), vec.end(), digit);
361
362     for (auto& elem : vec) {
363         std::cout.write(all_data.data() + elem.pos, elem.len);
364         std::cout << "\n";
365     }
366
367     return 0;
368 }

```

6 Тесты

Входные данные:

```

15.03.22 12345
01.01.23 7890
07.11.21 42
31.12.22 999999
22.02.22 777
09.05.23 2023
13.06.21 100500
25.10.22 8
03.07.23 55555
18.08.21 13

```

Выходные данные:

```

13.06.21      100500
18.08.21      13
07.11.21      42
22.02.22      777
15.03.22      12345
25.10.22      8
31.12.22      999999
01.01.23      7890
09.05.23      2023
03.07.23      55555

```

7 Вывод

Асимптотика по времени в среднем составляет $O((n + 2^{step}) \cdot m)$, где n — количество элементов, $step = 11$ — число обрабатываемых бит за одну итерацию, $m = \lceil 32/11 \rceil = 3$ —

количество итераций (разрядов). Таким образом, $O((n + 2048) \cdot 3) = O(n + 2048)$, что на практике сводится к $O(n)$ благодаря константному размеру диапазона. Асимптотика по памяти: $O(n)$, где n — число элементов, так как требуется дополнительный буфер того же размера для временного хранения. Реализация эффективно использует битовые операции и устойчивую сортировку подсчётом для каждого фрагмента, обеспечивая линейное время работы.